

Rapport de projet :

Conception d'un compilateur pour le langage AFSTAT

Auteurs :

Yasmina FARID
Yassine FADL
Lenny BORRY
Adrien CRISEO

Introduction 2
1. Analyse lexicale 3
1.1 Mots-clés du langage 3
1.2 Opérations statistiques 3
1.3 Types de données 4
1.4 Opérateurs et symboles 4
Opérateurs booléens 4
Opérateurs arithmétiques 4
Symboles de structure 4
1.5 Littéraux et identifiants 5
1.6 Gestion de fin de fichier, erreurs et commentaires 5
2. Analyse syntaxique 5
2.1 Structure générale d'un programme AFSTAT 5
2.2 Déclaration des sources 6
2.3 Déclarations des schémas et associations 6
Schémas 6
Association 7
2.4 Section des opérations 7
2.5 Jointure de données 7
2.6 Filtrage des données 8
2.7 Colonnes calculées 8
2.8 Expressions arithmétiques 9
2.9 Littéraux 9
2.10 Identifiants qualifiés 9
2.11 Blocs d'analyse 10
2.12 Opérations statistiques 10
3. Analyse sémantique 11
4. Génération de code C 12
5. Runtime d'exécution 12
6. Tests et démonstration 12

Organisation du dépôt 13
Chemins CSV 13
Validation de l'outil 13
Chaîne complète (end-to-end) 13
Tests fonctionnels (positifs) 14
Tests de robustesse (négatifs) 14
Couverture des tests 14
7. Limites et choix d'architecture 15

Introduction

Dans le cadre du projet de Compilation, nous avons réalisé une chaîne de compilation complète pour le langage dédié *AFSTAT*. En partant d'un programme déclaratif en langage de haut niveau, nous générons un code en *C* compilable et exécutable par un utilisateur.

En tant que *DSL*, ce projet permet à un utilisateur d'analyser et modifier des fichiers *CSV* de manière simplifiée grâce à des mécanismes de déclaration de sources de données, de schémas, de colonnes calculées, de filtrage, de jointures entre plusieurs sources, ainsi que des opérations statistiques classiques telles que la moyenne, la médiane, l'écart-type ou encore la génération d'histogrammes.

1. Analyse lexicale

Première étape du processus de compilation, l'analyse lexicale a pour rôle de lire chaque caractère du programme source *AFSTAT* et de les transformer en tokens qui seront ensuite exploités par l'analyse syntaxique.

1.1 Mots-clés du langage

Ce sont les mots-clés réservés au langage *AFSTAT,* correspondant aux instructions fondamentales qui décrivent les sources de données, leur structure et les traitements à effectuer.

Token	Lexème	Description
TOKEN_SOURCE	"source"	Déclare une source CSV
TOKEN_SCHEMA	"schema"	Déclare un schéma (colonnes + types)
TOKEN_ASSOCIATE	"associate"	Associe un schéma à une source
TOKEN_WITH	"with"	Mot de liaison dans associate
TOKEN_COMPUTE	"compute"	Ajoute une colonne calculée
TOKEN_ANALYZE	"analyze"	Ouvre un bloc d'analyses
TOKEN_JOIN	"join"	Déclare une jointure de datasets
TOKEN_INNER	"inner"	Spécifie un type de jointure (inner join)
TOKEN_FILTER	"filter"	Crée un dataset filtré
TOKEN_WHERE	"where"	Introduit une condition de filtrage

1.2 Opérations statistiques

Les opérations statistiques disponibles dans *AFSTAT,* utilisées lors de l'analyse de colonne afin de produire un résultat statistique.

Token	Lexème	Description
TOKEN_MEAN	“mean”	Calcul de la moyenne
TOKEN_MEDIAN	“median”	Calcul de la médiane
TOKEN_STDDEV	“std_dev”	Calcul de l’écart-type
TOKEN_MIN	“min”	Calcul du minimum
TOKEN_MAX	“max”	Calcul du maximum
TOKEN_HISTOGRAM	“histogram”	Calcul d’un histogramme
TOKEN_SUMMARY	“summary”	Résumé statistique complet

1.3 Types de données

Les types de données supportés par *AFSTAT* qui permettent de définir le type d’une colonne dans les schémas et de s’assurer de la compatibilité des opérations lors de l’analyse sémantique.

Token	Lexème	Description
TOKEN_INTEGER	“integer”	Type entier
TOKEN_FLOAT	“float”	Type flottant
TOKEN_STRING	“string”	Type chaîne de caractères

1.4 Opérateurs et symboles

Ces tableaux sont l’ensembles des opérateurs et symboles utilisés dans la construction d’expressions arithmétiques, de conditions de filtrage et de la structure globale des programmes.

Opérateurs booléens

Token	Lexème
TOKEN_EQ	“==”
TOKEN_NEQ	“!=”
TOKEN_LE	“<=”
TOKEN_GE	“>=”
TOKEN_LT	“<”
TOKEN_GT	“>”

Opérateurs arithmétiques

Token	Lexème
TOKEN_PLUS	“+”
TOKEN_MINUS	“-”
TOKEN_MUL	“*”
TOKEN_DIV	“/”

Symboles de structure

Token	Lexème	Description

TOKEN_LBRACE	“{”	Début de bloc
TOKEN_RBRACE	“}”	Fin de bloc
TOKEN_LPAREN	“(”	Parenthèse ouvrante
TOKEN_RPAREN	“)”	Parenthèse fermante
TOKEN_SEMI	“;”	Fin d’instruction
TOKEN_DOT	“.”	Accès qualifié (dataset.colonne)
TOKEN_ASSIGN	“=”	Affectation

1.5 Littéraux et identifiants

Ces expressions régulières sont essentielles pour utiliser les littéraux et identifiants du langage *AFSTAT*. Elles sont utilisées pour les noms de constantes, de colonnes, de schémas et de sources.

Token	Expression régulière	Valeur associée
TOKEN_INT_LIT	[0-9]+	yyval.int_val
TOKEN_FLOAT_LIT	[0-9]+"."[0-9]+	yyval.float_val
TOKEN_STRING_LIT	\"([^\\"]|\\.)*\"	yyval.str
TOKEN_IDENT	[A-Za-z_][A-Za-z0-9_]*	yyval.str

1.6 Gestion de fin de fichier, erreurs et commentaires

La fin du fichier est marqué par le token *<<EOF>>,* on retourne la valeur 0 au parser.

Tout caractère inattendu est une erreur lexicale et on retourne le token *YYUNDEF* ce qui permet au compilateur de signaler l’erreur avec un message précis.

On ignore totalement les lignes débutant par “*//*” ainsi que les espaces et retours à la ligne.

2. Analyse syntaxique

Faisant suite à l’analyse lexicale, l’analyse syntaxique s’assure que la liste de tokens produite par le lexer corresponde à la grammaire du langage. Le parser reconnaît alorsla structure globale d’un programme, l’enchaînement des instructions, ainsi que l’organisation interne des expressions, des déclarations et des blocs d’analyse.

2.1 Structure générale d’un programme *AFSTAT*

Un programme *AFSTAT* est composé de quatre parties principales:

1. Une section de sources obligatoire

2. Une section de déclarations obligatoire
3. Une section d'opérations optionnelle
4. Une section d'analyse obligatoire

La grammaire générale du programme est définie ainsi:

```
program
        : source_section declaration_section operation_section analyze_section
```

2.2 Déclaration des sources

Unprogramme commence par au moins une directive source qui permet de déclarer les fichiers CSV utilisés.

```
source_section
        : source_section source_decl
        | source_decl

source_decl
        : TOKEN_SOURCE TOKEN_IDENT TOKEN_STRING_LIT TOKEN_SEMI
```

2.3 Déclarations des schémas et associations

Les schémas et les associations peuvent s'alterner , ils partagent donc ce bloc.

```
declaration_section
        : declaration_section declaration
        | declaration

declaration
        : schema_decl
        | association_decl
```

Schémas

Un schéma définit la structure logique des données associées à une source. Il est composé d'un identifiant et d'une liste de champs typés.
Lors de l'analyse syntaxique, chaque schéma est enregistré dans la table globlale des schémas.
On déclare une erreur lors d'un dépassement du nombre maximal de schémas ou si un schéma est rédéfini.

```
schema_decl
        : TOKEN_SCHEMA TOKEN_IDENT TOKEN_LBRACE schema_field_list
TOKEN_RBRACE

schema_field_list
    : schema_field_list schema_field
    | schema_field

schema_field
    : TOKEN_IDENT type_token TOKEN_SEMI

type_token
    : TOKEN_INTEGER
    | TOKEN_FLOAT
    | TOKEN_STRING
```

Association

L'instruction associate permet de rattacher un schéma à une source de données déclarée précédemment.

```
association_decl
        : TOKEN_ASSOCIATE TOKEN_IDENT TOKEN_WITH TOKEN_IDENT TOKEN_SEMI
```

Cette étape permet d'établir le lien entre la structure logique des données et le fichier CSV correspondant.

2.4 Section des opérations

La section des opérations est optionnelle et peut contenir plusieurs instructions successives, chacune produisant éventuellement un nouveau dataset virtuel.

```
operation_section
        : operation_section operation
        | /* Vide */

operation
        : join_stmt
        | filter_stmt
        | compute_stmt
```

2.5 Jointure de données

L'instruction join permet de créer un nouveau jeu de données en effectuant une jointure interne entre deux sources ou datasets existants.

```
join_stmt
        : TOKEN_JOIN TOKEN_IDENT TOKEN_ASSIGN join_source TOKEN_INNER
TOKEN_JOIN join_source TOKEN_SEMI

join_source
        : TOKEN_IDENT TOKEN_DOT TOKEN_IDENT
```

Les deux colonnes utilisées pour la jointure doivent exister et être de types compatibles. Le schéma résultant est automatiquement construit en combinant les colonnes des deux schémas d'origine.

2.6 Filtrage des données

L'instruction filter crée un nouveau dataset à partir d'un dataset existant en appliquant une condition booléenne.

```
filter_stmt
        : TOKEN_FILTER TOKEN_IDENT TOKEN_ASSIGN TOKEN_IDENT TOKEN_WHERE
condition TOKEN_SEMI
```

La condition de filtrage doit être une expression valide de type booléen.

```
condition
   : expression TOKEN_EQ expression
   | expression TOKEN_NEQ expression
   | expression TOKEN_LT expression
   | expression TOKEN_GT expression
   | expression TOKEN_LE expression
   | expression TOKEN_GE expression
```

2.7 Colonnes calculées

L'instruction compute permet d'ajouter une nouvelle colonne calculée à un dataset existant.

```
compute_stmt
        : TOKEN_COMPUTE TOKEN_IDENT TOKEN_ASSIGN expression TOKEN_SEMI
```

L'expression associée doit être de type numérique. Le type résultant est automatiquement déduit (entier ou flottant).

2.8 Expressions arithmétiques

Les expressions arithmétiques respectent les règles de priorité classiques et supportent le moins unaire.

```
expression
        : expression TOKEN_PLUS term
        | expression TOKEN_MINUS term
        | term

term
        : term TOKEN_MUL factor
        | term TOKEN_DIV factor
        | factor

factor
        : TOKEN_LPAREN expression TOKEN_RPAREN
        | literal
        | qualified_ident
        | TOKEN_MINUS factor %prec UMINUS
```

2.9 Littéraux

Chaque littéral produit une expression typée et directement traduisible en code C.

```
literal
        : TOKEN_INT_LIT
        | TOKEN_FLOAT_LIT
        | TOKEN_STRING_LIT
```

2.10 Identifiants qualifiés

Les colonnes peuvent être référencées soit directement, soit de manière qualifiée par leur dataset.

```
qualified_ident
        : TOKEN_IDENT TOKEN_DOT TOKEN_IDENT
        | TOKEN_IDENT
```

Ces références sont vérifiées lors de l'analyse afin de garantir l'existence des colonnes et leur accessibilité.

2.11 Blocs d'analyse

Le langage impose la présence d'au moins un bloc analyze, définissant les opérations statistiques à effectuer.

```
analyze_section
        : analyze_section analyze_block
        | analyze_block

analyze_block
        : TOKEN_ANALYZE TOKEN_IDENT TOKEN_LBRACE analyze_item_list
TOKEN_RBRACE
```

Le dataset ciblé doit être déclaré ou correspondre au dernier dataset produit.

2.12 Opérations statistiques

Chaque bloc analyze contient une liste d'opérations statistiques appliquées à des colonnes spécifiques.

```
analyze_item_list
        : analyze_item_list analyze_item
        | analyze_item

analyze_item
        : analyze_op TOKEN_IDENT TOKEN_SEMI

analyze_op
        : TOKEN_MEAN
        | TOKEN_MEDIAN
        | TOKEN_STDDEV
        | TOKEN_MIN
        | TOKEN_MAX
        | TOKEN_HISTOGRAM
        | TOKEN_SUMMARY
```

Des vérifications sémantiques sont effectuées pour garantir la compatibilité entre l'opération demandée et le type de la colonne analysée.

3. Analyse sémantique

L'analyse sémantique d'AFSTAT repose sur une **table des symboles globale** et sur des **vérifications effectuées au fil du parsing**.
La table des symboles (symtable.c / symtable.h) centralise toutes les informations déclaratives du programme : **sources**, **schémas**, **associations**, ainsi que **colonnes et types**. Elle permet notamment de détecter les **redéclarations**, les dépassements de capacité et d'assurer la **résolution des références** de type dataset.colonne.

Les contrôles sémantiques sont déclenchés lors de l'analyse des instructions dans parser.y :

- Pour les déclarations source, schema et associate, le compilateur vérifie l'unicité des identifiants, l'existence des entités référencées (schéma ou source), ainsi que la cohérence des associations.
- Pour join, il vérifie la présence des colonnes dans leurs schémas respectifs, puis la compatibilité des types (numérique avec numérique, chaîne avec chaîne). Le schéma du dataset résultat est ensuite construit automatiquement à partir des colonnes combinées.
- Pour filter, il vérifie que le dataset source est connu et que la condition est bien de **type booléen**. Le schéma du dataset filtré est obtenu par clonage du schéma source.
- Pour compute, un dataset courant doit être défini, l'expression doit être de type **numérique**, puis une nouvelle colonne est ajoutée au schéma avec **type déduit** (int ou float selon promotion).
- Pour analyze, la cible est résolue vers un schéma valide (schéma déclaré ou association existante), la colonne doit exister, et l'opération doit être compatible avec son type : mean, median, std_dev, min, max, summary s'appliquent aux colonnes int/float, tandis que histogram est autorisé sur tous les types.

Enfin, le typage des **expressions** applique une promotion simple int → float en cas d'opération mixte, et rejette les combinaisons incohérentes (ex. string + int ou comparaisons interdites). Toute référence à un identifiant non défini (dataset ou colonne inconnue) déclenche une erreur explicite et arrête la compilation.

Limite de l'architecture : le compilateur ne génère pas de code intérmediaire par faute de temps après échec du débuggage. Les opérations reconnues sont stockées en mémoire sous forme de structures de requêtes (ex. ComputeRequest, FilterRequest,

JoinRequest, AnalyzeRequest) ainsi que des représentations d'expressions (CodeExpr) exploitées directement par le backend.

4. Génération de code C

La génération de code produit un fichier C complet à partir des informations collectées lors de l'analyse (table des symboles + requêtes enregistrées + expressions typées). La fonction principale de sortie est déclenchée dans parser.y via la logique d'écriture du fichier final (sortie par défaut ou via -o <file>).

Le fichier C généré inclut le runtime (runtime.h) et contient un programme complet qui :

1. déclare une structure RtTable par dataset/schéma,
2. charge les fichiers CSV correspondants via rt_load_csv, avec validation de l'en-tête et des types,
3. exécute séquentiellement les transformations : **join** (construction d'un nouveau dataset), **filter** (copie conditionnelle), **compute** (calcul de colonnes via CodeExpr),
4. exécute ensuite les opérations d'analyse (rt_mean, rt_histogram, rt_summary, etc.) et affiche les résultats avec printf,
5. libère la mémoire associée aux tables runtime en fin d'exécution.

Le compilateur propose également une interface en ligne de commande : -version, -tos, -syntax-only et -o <file> pour choisir le nom du fichier C produit.

5. Runtime d'exécution

L'exécution du C généré s'appuie sur une bibliothèque runtime (runtime.c / runtime.h) qui fournit les primitives nécessaires pour manipuler les données tabulaires. Elle inclut notamment : le chargement et la validation de CSV (rt_load_csv), une représentation tabulaire typée (RtTable), des opérations utilitaires (copie de lignes/colonnes, helpers de join et filter) et des fonctions de statistiques numériques (moyenne, médiane, écart-type, min/max), ainsi que la gestion d'histogrammes pour des colonnes non numériques.

Le runtime supporte un typage simple basé sur trois types principaux : **int**, **float** et **string**.

6. Tests et démonstration

Le projet inclut un jeu de tests permettant de valider à la fois la construction de l'outil, les fonctionnalités du langage AFSTAT et les erreurs obligatoires à détecter avant génération/exécution.

Organisation du dépôt

lexer.l, parser.y : analyse lexicale et syntaxique (+ sémantique et capture des requêtes).

symtable.[ch], ast.[ch] : table des symboles, AST minimal.

runtime.[ch] : runtime (tables, CSV, stats, join/filter/compute).

tokens.h : énumération des tokens.

exemples/ : prog1..prog7 + CSV dans **exemples/data/.**

test/ : tests positifs/négatifs, **test_cmds.txt** (commandes).

Chemins CSV

Les **.afs** pointent sur **data/<csv>** ; exécution depuis **exemples/** ou avec le chemin relatif correct.

Validation de l'outil

- Reconstruction complète : make clean && make
- Build + tests parser : make clean && make && make test
- Génération + compilation + exécution de tous les exemples : make test_codegen RUN=1
- ou, pour un fichier : ./afstat_parser -o gen.c exemples/progX.afs && gcc -Wall -Wextra -std=c11 -o gen gen.c runtime.c -lm && (cd exemples && ../gen)
- Options CLI :
    - version : affiche les membres du projet.
    - tos : dump de la table des symboles après parsing.
    - syntax-only : arrêt après analyse syntaxique (pas de code généré).
    - -o <file> : écrit le C généré dans <file> (par défaut afstat.c).

Chaîne complète (end-to-end)

- Génération + compilation + exécution : ./afstat_parser -o /tmp/demo.c exemples/prog1.afs && gcc ... && /tmp/demo_bin
- Démonstration sur plusieurs programmes : RUN=1 make test_codegen (prog1 à prog5)

Tests fonctionnels (positifs)

- make test_positive_simple : programme minimal (source/schema/associate/analyze)
- make test_positive_compute : ajout de colonne calculée
- make test_positive_join : jointure inner join
- make test_positive_join_compute : join suivi de compute
- make test_positive_filter : filtrage numérique
- make test_positive_filter_string : filtrage sur chaîne de caractères
- make test_positive_histogram : analyse histogramme

Tests de robustesse (négatifs)

- Erreur lexicale : make test_lexical_invalid
- Erreur syntaxique : make test_syntax_error
- Identifiant/colonne inconnue : make test_semantic_unknown_column, make test_undefined_identifier
- Join incompatible : make test_semantic_join_mismatch
- Analyse incompatible : make test_analyze_type_mismatch
- Expression invalide : make test_expr_type_mismatch
- Schéma incohérent avec CSV : make test_schema_type_mismatch, make test_bad_schema

Couverture des tests

- Prog1..prog7 passent en génération/exec.
- make test valide le parser.
- make test_codegen RUN=1 exécute les binaires générés (moyennes, histogrammes, summaries affichés).
- Dossier test/ : jeux positifs/négatifs (lexical, syntaxe, colonnes inconnues, join incompatible, analyze type mismatch, etc.).

Tous les tests positifs affichent des résultats à l'écran (moyennes, histogrammes, résumés), ce qui valide l'exécution complète du programme généré.

7. Limites et choix d'architecture

Le compilateur AFSTAT est conçu comme une chaîne simple et directe : il n'y a pas de génération d'IR 3-adresses, on génère directement du C (considéré comme code intermédiaire avant compilation gcc) et il n'y a pas de passes d'optimisation. Le backend cible uniquement du **C** accompagné d'un runtime dédié(runtime.c, runtime.h) qui est simple ne prenant en charge que *int, float* et *string*. Les jointures sont implémentées de manière naïve (double boucle), ce qui privilégie la simplicité et la clarté à la performance.